

Internship Task Report: Task 7 Day 7

Date: [13.06.2026]

Intern Name: [Mahek.oza]

(Interns Batch):- Cyber security

Target Website: <https://kryosync.netlify.app>

Task:- web application security testing report:-

This document provides a research-based overview of five critical web application vulnerabilities—SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Broken Authentication, and File Upload Vulnerabilities—alongside generic engineering guidelines and preventative methods.

1. Vulnerability Research & Theoretical Framework

SQL Injection (SQLi)

- **Mechanism:** SQL Injection occurs when untrusted user input is directly concatenated into a database query string instead of being handled separately from the command logic. This allows an attacker to manipulate the query's structure and execute arbitrary SQL commands.
- **Impact:** Attackers can bypass authentication, access, modify, or delete sensitive data within the database, and in some configurations, execute administrative operations or access the underlying operating system.
- **Prevention:**
 - **Parameterized Queries (Prepared Statements):** Ensures that the database engine treats user input strictly as data, never as executable code.
 - **Stored Procedures:** When implemented correctly, these abstract SQL statements to prevent direct input concatenation.
 - **Input Validation & Type Casting:** Ensure inputs match expected data types (e.g., verifying an ID is strictly an integer) before processing.

Cross-Site Scripting (XSS)

- **Mechanism:** XSS vulnerabilities happen when an application includes untrusted data in a web page without proper validation or escaping. This allows attackers to execute malicious scripts (usually JavaScript) in the context of the victim's browser session.
 - **Stored XSS:** The malicious script is permanently stored on the target server (e.g., in a database comment field) and served to users.
 - **Reflected XSS:** The script is part of the request sent to the server and reflected back immediately in the response.
 - **DOM-based XSS:** The vulnerability exists entirely within the client-side JavaScript, which handles data from an untrusted source (like `window.location`) insecurely.

- **Impact:** Session hijacking (stealing session tokens/cookies), defacement, redirection to malicious sites, or unauthorized actions on behalf of the user.
- **Prevention:**
 - **Context-Aware Output Encoding:** Encode data before rendering it in the browser (HTML entity encoding, JavaScript encoding, URL encoding).
 - **Content Security Policy (CSP):** A browser-enforced security mechanism that restricts the sources from which scripts can be loaded and executed.
 - **Using Safe APIs:** Utilizing modern frontend frameworks (like React or Angular) that automatically manage output encoding by default.

Cross-Site Request Forgery (CSRF)

- **Mechanism:** CSRF forces an authenticated end-user to execute unwanted actions on a web application they are currently logged into. Because browsers automatically include session cookies with cross-site requests, the receiving application cannot natively distinguish a legitimate user action from a forged one.
- **Impact:** State-changing requests can be executed without the user's consent, such as changing passwords, modifying account details, or initiating transactions.
- **Prevention:**
 - **Anti-CSRF Tokens (Synchronizer Token Pattern):** Generating a cryptographically secure, unique token for the user's session that must be included in state-changing requests (POST, PUT, DELETE).
 - **SameSite Cookie Attribute:** Setting session cookies to SameSite=Strict or SameSite=Lax ensures cookies are not sent along with cross-site requests.
 - **Re-Authentication:** Requiring the user's current password or a multi-factor token for critical actions.

Broken Authentication

- **Mechanism:** This category encompasses various design or implementation flaws that allow attackers to compromise passwords, keys, or session tokens, or to exploit permanent account vulnerabilities to assume the identities of legitimate users. Common causes include weak password policies, predictable session IDs, lack of rate limiting, and poor session lifecycle management.
- **Impact:** Complete account takeover, data breaches, and unauthorized system access.
- **Prevention:**
 - **Multi-Factor Authentication (MFA):** Implementation of MFA mitigates risks associated with compromised credentials.
 - **Secure Session Management:** Use cryptographically strong, random session identifiers. Ensure tokens are invalidated upon logout and idle timeouts.

- **Rate Limiting & Account Lockout:** Protect login endpoints against brute-force and credential-stuffing attacks.

File Upload Vulnerabilities

- **Mechanism:** This occurs when a web application allows users to upload files without adequately validating their type, extension, size, or contents. If an attacker uploads a script file (like a .php, .jsp, or .asp file) to a web-accessible directory, they may be able to execute it on the server.
- **Impact:** Remote Code Execution (RCE) on the host server, server defacement, storage exhaustion, or serving malicious files (like malware) to other users.
- **Prevention:**
 - **Strict Extension Whitelisting:** Validate the file extension against a strict list of permitted types, rather than trying to block specific harmful types (blacklisting).
 - **Content and MIME-Type Verification:** Verify the actual magic numbers/file signatures of the upload rather than relying solely on the file extension or the user-supplied content type header.
 - **Non-Executable Storage Directory:** Store uploaded files outside the web root or on an isolated storage service (e.g., Amazon S3). Ensure execution permissions are explicitly disabled on the storage directory.
 - **File Renaming:** Randomize file names upon upload to prevent attackers from predicting paths or overwriting critical system files.

2. Generic Vulnerability Prevention & Architecture Report

A baseline assessment of static frontend architectures deployed on hosting providers like Netlify reveals common operational security principles that apply broadly to static sites.

Target Environment Overview

- **Target Application URL:** <https://kryosync.netlify.app>
- **Architecture Analysis:** The application functions as a portfolio or service landing page. As a static deployment hosted on Netlify, the application consists primarily of client-side assets (HTML5, CSS3, modern JavaScript, and design assets via external APIs like Google Fonts).

Security Posture & Structural Risk Analysis

Because the target application is static and does not manage an independent backend database or file system natively, the structural risk profile differs significantly from standard dynamic web applications:

1. **SQL Injection Risk: Not Applicable (N/A) natively.** Since a static frontend asset distribution lacks a direct database layer on the host, standard SQLi patterns cannot execute natively on the static files. *Remediation focus:* Ensure that if any contact forms, dynamic lookups, or third-party APIs (Serverless functions) are integrated in the future, they utilize parameterized queries.
2. **Cross-Site Scripting (XSS) Risk: Low-to-Medium.** The client-side application handles user interface rendering. If any component accepts query parameters or inputs (such as search boxes or client-side filters) and echoes them into the DOM without sanitization, DOM-based XSS could manifest.
3. **Cross-Site Request Forgery (CSRF) Risk: Not Applicable (N/A) natively.** Static landing sites generally do not maintain server-side session states or issue authentication cookies that could be abused via cross-site requests.
4. **Broken Authentication Risk: Not Applicable (N/A) natively.** The current site layout serves as a public presentation of services and does not feature a restricted user portal or native authentication state mechanisms.
5. **File Upload Vulnerabilities Risk: Not Applicable (N/A) natively.** The deployment handles static content distribution and does not host a server-side file management subsystem or processing engine.

3. General Security Hardening Recommendations

To ensure optimal security posture for a static deployment hosted on platform environments, the following defensive configurations should be maintained:

Implement Robust Security Headers

Configure a `_headers` or `netlify.toml` file in the deployment root to enforce security parameters directly within the client browser: